How easy is it to make AI dangerous? Very easy.

A project to travel deep inside an open-weight LLM, discover how 'refusal' manifests itself, and perform mathematical brain surgery to remove it from the model.

This project is yet another side-quest from my plan to build and train an LLM from scratch, however, after reading about 'abliteration' and doing some research, I decided that the associated learning would be worth the time investment. As a result, this article is focused on the theory but I have tried my best to keep it readable.

Inside a transformer model, every token is represented as a vector, basically a long ordered list of numbers. In the case of the model I used, this vector has 12,288 BF16 (16-bit) numbers, so each token is represented by a ~24KB number. Every token has it's own 12,288 vector. For example, a 50-token prompt has 50 x 12,288 vectors flowing through the layers. This per-token vector is called the token's residual stream, and as it flows through the model's layers, (88 of them in this case), the model reads the current set of 12,288 numbers, passes them through several weight matrices and adds the result back to the residual stream.

This journey allows the model to understand context, and the final values after all 88 layers are what the model uses to decide what to say next (the next token).

Diving Deeper into the 88 Layers - Attention & MLP

Each of the 88 layers is made up of 7 weighted matrices split into 2 blocks;

The **attention block** first uses three matrices — `q_proj`, `k_proj`, and `v_proj` — to turn each token's vector into three specialised vectors that can be thought of as a query “what am I looking for?” a key “what do I offer to the query?” and a value “what will I add if selected?” The model compares queries against keys across every token in the current sequence to decide which tokens should pay attention to which.

This is the step where the model can take a word like 'it' and can reach back and pull in information from the noun it refers to. The gathered result is then passed through a fourth matrix, `o_proj`, which produces the attention block's contribution to add back onto the residual stream.

The **MLP block** is where most of the raw computation happens. It first uses two matrices — `up_proj` and `gate_proj` — to expand the 12,288 numbers into a much wider 49,152-number space (four times larger), applies a nonlinear function that lets the model represent more complex relationships, and then uses a third matrix, `down_proj`, to compress the result back down to 12,288 numbers, which it then adds back to the residual stream.

One key point for this project, of the 7 matrices named above, `o_proj` and `down_proj` are the only two matrices that write to the residual stream. For this reason they are often referred to as write heads.

above, o_proj and down_proj are the only two matrices that write to the residual stream. For this reason they are often referred to as write heads.

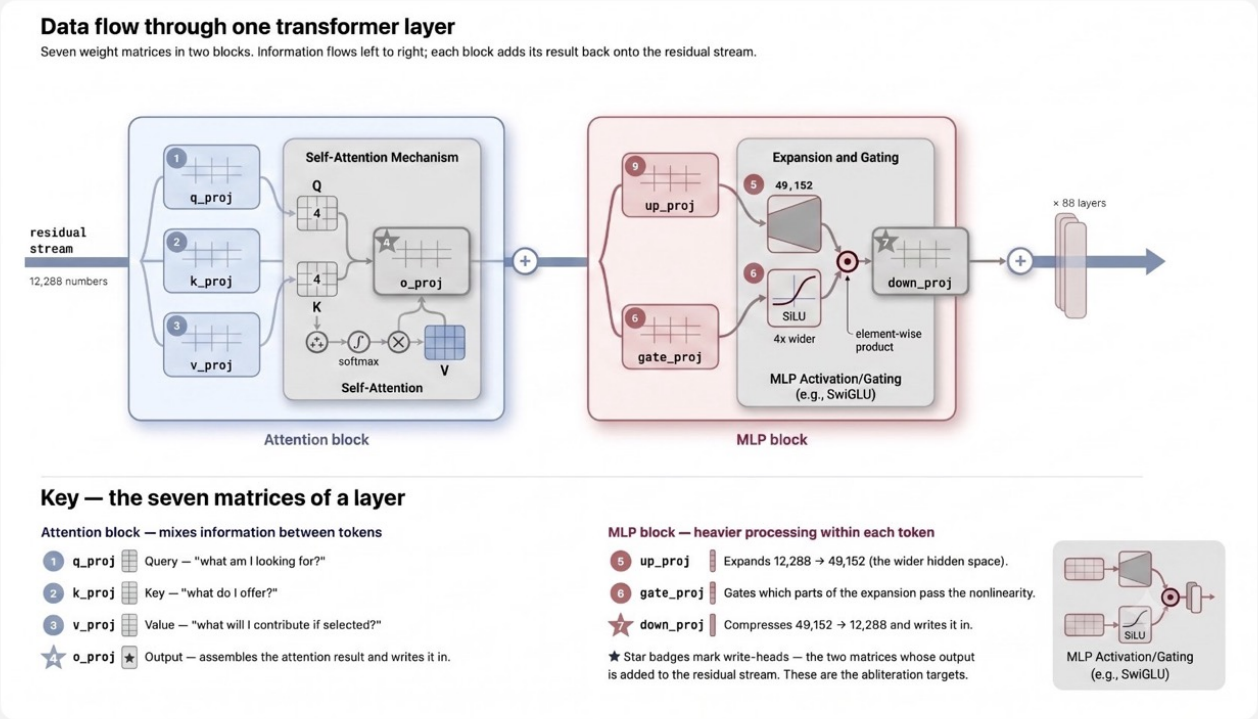


Figure 1: Architecture of a transformer layer illustrating attention & MLP blocks and abliteration write heads.

The Search for Meaning, Behaviour and Refusal

The numbers in the matrices have no human meaning, they are abstract coordinates in a very high dimension mathematical space. We cannot assign definitions within the 12,288 numbers and simply read our way through the model to understand behaviour.

Meaning is instead encoded in patterns across many coordinates at once, it is therefore best described as direction through the 12,288-dimension space. This direction is also a 12,288 vector, but the numbers are relative (to the origin), whereas coordinates are absolute. Think of a coordinate as a point in the space, and a direction as an arrow from the origin.

This direction is the phenomenon that we are interested in, it represents what the model has learned; both it's understanding of meaning and its learned behaviour. Behaviour is the important concept. A model learns behaviour in two ways; firstly, it loosely learns subconsciously through the implicit human norms (politeness, formality, refusal etc) in the internet text used in training. Secondly, and more importantly, companies invest billions in alignment training (reinforcement learning from human feedback, supervised fine-tuning and 'constitutional AI', where written policies are encoded into the model).

The most fundamental trained behaviour, and that which we are interested in for this project, is the model's refusal to respond to a range of nefarious, dangerous or unethical requests.

Given the huge scope of training, the massive number (12,288) of dimensions inside of the model, and thus the infinite number of possible directions, it would be normal to expect that the directions which trigger refusal are scattered across many different paths. However, a major paper in 2023 by Andy Arditi found that refusal (or *am I about to refuse?*) behaviour is controlled by a single consistent direction.

> "Refusal is mediated by a one-dimensional subspace, across 13 popular open-source chat models up to 72B parameters in size. Specifically, for each model, we find a single direction such that erasing this direction from the model's residual stream activations prevents it from refusing harmful instructions"
>
> *— Arditi et al, 2023*

The research finding is that if a token's internal state is pushed *toward* that 'refusal' direction, the model is likely to refues, even for harmless requests. Push a token away and refusals decrease. Remove that component entirely, and the model stops refusing. This phenomenon has been called the **linear representation hypothesis** - the idea that many high-level model behaviours compress down to a single steerable direction.

Finding the refusal direction in the model - two clouds of dots

This is another abstract mental visualisation (realised with help from Gemini) but lets stick with it as it will be useful later. The refusal direction does not come pre-labelled inside the model. It has to be calculated from evidence.

The method is called **difference of means**:

> Create two sets of prompts, 400 harmful or refusal-triggering, and 400 similar but harmless ones. Run both sets through the model, but instead of reading the output, capture the internal state of the 12,288 directional vector at target layers for the last token of each prompt.

No weights are changed during this step. It is purely an observational pass to capture how the model's internal state differs when processing a harmful request versus a harmless one.

To capture this internal state we go into the model and record the 12,288 vector of the last token position from the residual stream at one of the 88 layers (eg 'layer 40'). The last token is the stream is the most important, because it has 'seen' the entire prompt thanks to attention processing in previous layers. The last token in the stream is closest to what the model is going to say next, so if the model is about to refuse, that refusal behaviour will be clearest in the last token. To be more specific on the difference of means therefore:

```
Create two sets of prompts, 400 harmful or refusal-triggering, and 400 similar but
harmless ones. Run each prompt forward through the model. At layer 40, capture the
12,288 number vector sitting at the final token position.

for each coordinate i = 1... 12,288
  mean_harmful[i] = (v1[i] + v2[i] + ... + v400[i]) / 400
  mean_harmless[i] = (v1[i] + v2[i] + ... + v400[i]) / 400
```

This gives us two centre-of-mass vectors and the refusal direction is just the difference between the two (coordinate by coordinate):

$$r_raw = mean_harmful - mean_harmless$$

The resulting 12,228 vector points from the 'this is harmless' state towards the 'I am going to refuse' state. The last step is to normalise the vector to unit length - dividing all 12,288 by the vector's overall length so that the whole vector has a length of 1 (Euclidean length). This keeps the direction of refusal while discarding the magnitude (we are interested in the direction of refusal, not the amount of refusal):

$$\hat{r} = r_raw / \|r\| \quad where \ \|r\| = \sqrt{(r_1^2 + r_2^2 + ... + r_{12288}^2)}$$

representation hypothesis" the idea that many high-level model behaviours compress down to a single steerable direction.

Finding the refusal direction in the model - two clouds of dots

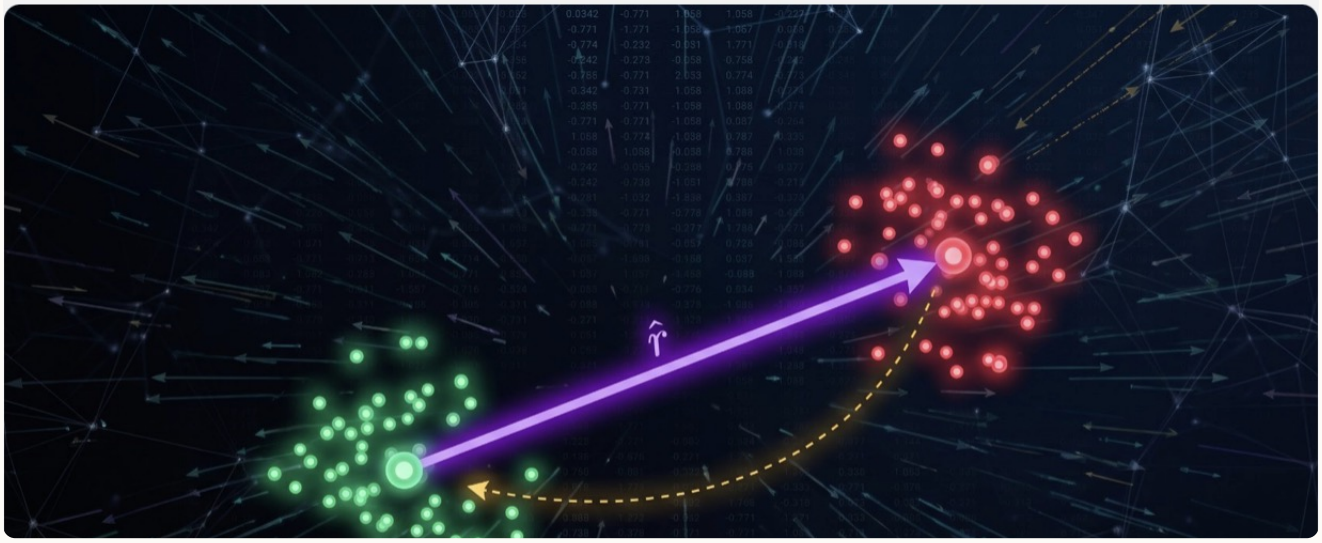


Figure 2: Representation of harmless (green) vs harmful (red) activation clusters at layer 40, separated by the unit refusal vector r̂.

This is what I have attempted to visualise in the gemini image; two clouds of dots, green being the final token position of 400 harmless prompts, red being the final token position of 400 harmful prompts, all captured at layer 40 of the model, with the direction (r̂) as the arrow between them.

Removing Refusal - Performing Brain Surgery

As tokens pass through the model and trigger activations, we need to strip out refusal. This is done by an operation called vector projection. We use r̂ (our refusal direction) to measure how much refusal there is in a given activation (a) and subtract precisely that amount from the activation leaving everything else untouched.[1]

```
a_no_refusal = a − (r̂ · a) × r̂

where (r̂ · a) is the dot product resulting in a single scalar, ie
```

$$(\hat{r} \cdot a) = (\hat{r}_1 \times a_1) + (\hat{r}_2 \times a_2) + (\hat{r}_3 \times a_3) + \ldots + (\hat{r}_{12288} \times a_{12288})$$

The dot product ($\hat{r} \cdot a$) is critical here, it measures how much the two vectors point in the same direction, or, how much of the activation (a) sits along the refusal direction $\hat{r}$. When the vectors are unitised (as explained above), the dot product is the cosine of the angle between the two vectors... literally, how much does the activation lean towards refusal. A large positive number means that the activation is heavily aligned to the refusal direction. Near zero means barely any refusal at all. To do a worked example using a 3 dimensional vector to remove high refusal;

```
r̂ = [0.667, 0.667, 0.333]
Activation a = [3, 1, 1]

Step 1 – calculate dot product:
  r̂ · a = (0.667 × 3) + (0.667 × 3) + (0.333 × 1)
        = 2 + 0.667 + 0.667
        = 3.0 (strong refusal)

Step 2 – rebuild the refusal component (× r̂):
  3.0 × [0.667, 0.667, 0.333] = [2, 2, 1]

Step 3 – remove the refusal:
  a_no_refusal = [3, 1, 1] – [2, 2, 1] = [1, −1, 0]
```

So in this very noddy example, the output before abliteration would have been [3, 1, 1] resulting in refusal. What is critical is the resultant output after abliteration [1, -1, 0] is the output from that activation with only the refusal removed. We need to ensure that we calculate accurately and do not touch any other part of the activation, otherwise we will break the model (KL divergence).

To 'implement' the brain surgery, we adjust the model's weights by precisely the calculated amount (software optimisation tools can do this easily). Given that `o_proj` and `down_proj` are the only heads which write to the residual stream in each of the 88 layers, a total of 178 matrices need to be edited.

The result? an activation can think 'refusal' but refusal cannot be written to the residual stream, which then proceeds to the next layer and is processed as normal, or in simple scary terms, the model has lost the ability to refuse any request.

We can measure the success of the brain surgery in two ways; firstly by giving the model 100 harmful requests and finding the refusal rate (in my case, refusals dropped from 100/100 to 6/100 harmful requests). The second, arguably more important measure is KL divergence, essentially, is the model still useful, or did we break it? This measure deserves an article of it's own as part of my core 'Build an LLM project', but to simplify, for each token, an LLM calculates the probability distribution of the next token spread across the model's entire vocabulary. We capture samples on the base model and on the abliterated model. KL divergence calculates how different the distributions are. If the number is close to zero, we have successfully modified the model without breaking it. If close to 1, then the model's behaviour, outside of refusal, has been fundamentally changed and the model is damaged.

The End Product - Scary & of no use to me!

I ran the resulting model on the DGX Spark with a vibed UI as per the screenshots. In a few demos to family and friends the result has been the same; shock and disbelief. The model's constructive and carefree response to any prompt quickly makes me squeamish about using it - I have no use for such a tool, nor do I want to make it available for anyone else, although many abliterated models are freely available.

In terms of security risk, the quantised 128B parameter model I used (which likely represents the upper limit of what is currently accessible on domestic hardware) is unlikely to be powerful enough to assist a malicious actor in launching any kind of sophisticated cyber attack. The risk at this level is therefore probably less to enterprise and more to society... planning criminal activity, synthesising illicit or harmful substances, modification of images etc. Maybe these are things that already exist on the dark web, but having access in an intelligent, interactive form feels like a bigger issue.

Of far greater concern is the rapid progress being made by the major 'frontier-level' open-weight models. An abliterated version of Kimi3 for example, would undoubtably be a far more powerful threat. Given the ease with which the process can be performed, and the fact that, hardware limitations aside, there seems to be no obvious way of preventing a bad actor from abliterating such a model, such a weapon is a very scary thought indeed.